
教育部高等学校软件工程专业教学指导委员会
软件工程专业推荐教材
高等学校软件工程专业系列教材
现代软件工程基础
彭鑫 游依勇 赵文耘◎编著
教学课件
教学视频
示例代码
课程实践
清华大学出版社

第七章 软件体系结构

软件体系结构（Software Architecture）

- 对于大规模软件系统，特别是在网络上部署的分布式软件系统而言，更高层的软件体系结构（软件架构）设计是必不可少的
- 软件系统的高层设计结构，决定了系统的高层分解结构及其他一些最重要的设计决策
  - ✓ 为满足软件系统整体的功能性需求、质量需求和约束（如项目完成时间、需要符合的标准等）提供了一种解决方案
  - ✓ 为后续更加具体的组件级设计（类似于楼层和房间内部设计）和实现（类似于建筑施工）提供了指导和参考

即使是在倡导增量、迭代开发的敏捷开发过程中，也应该在早期关注于整体的体系结构设计，因为体系结构很难以增量的方式进行开发，而根据变化重构体系结构是十分昂贵的（与之相比较，根据变化重构单个组件相对容易）（Sommerville，2018）。

软件体系结构的内容

- 一组软件组件：表示软件系统高层分解的结构元素，每个组件可以是简单的程序模块或类，或复杂的子系统以及扩展的数据库、中间件等
- 软件组件的外部属性：功能性属性（如接口、交互协议等）以及非功能性质量属性（如响应时间、吞吐量等）等组件外部属性
- 软件组件之间的关系：不同组件之间的静态依赖、动态交互以及运行时部署等关系
- 全局的实现约定：涉及多个组件、需要在系统全局作出约定的技术决策，例如实现语言、异常处理策略、共享资源使用方式、命名方式等

软件体系结构的作用

- 初步确认设计方案是否能有效满足需求
- 为满足未来演化和复用的需要而做出规划
- 多种候选技术方案对比选择的依据
- 识别并降低软件实现的风险
- 软件开发任务分工协作的基础
- 相关涉众沟通和交流的基础

软件体系结构决策

- 软件体系结构设计与建筑、汽车设计等其他设计过程一样，都是一种复杂的构思和决策过程
- 在此过程中，设计师需要考虑很多问题，而每个问题都可能存在多种候选的方案
- 设计是一个搜索过程，搜索空间是一个巨大的由各种设计问题候选方案组成的组合空间
  - ✓ 目标与条件：需要满足的一组目标以及相关的条件
  - ✓ 效用函数：针对每一项目标或条件的评估函数，能够根据目标或条件的满足情况计算出效用值
  - ✓ 约束：设计方案所需要满足的约束，其中有些约束是绝对的（即满足或不满足），而有些约束则具有一定的弹性（存在一定的协商空间）

软件体系结构决策的几个方面

- 目标与条件
  - ✓ 总体目标一般比较抽象和宏观，需要不断分解得到更加具体的子目标和要求的条件
  - ✓ 例如，校园一卡通系统中“使用方便快捷”这一高层目标可以不断细化得到“刷卡操作简单”、“刷卡响应时间快”等具体目标以及“刷卡消费结果在1秒内返回”等条件
- 效用函数
  - ✓ 针对单项目标或条件的评估函数一般都不是线性的，而是以渐进曲线的方式趋于饱和
  - ✓ 例如，刷卡消费功能的响应时间从10秒提升到1秒时效用值显著提高，但从1秒提升到0.1秒效用值已经很难显著提升了
- 约束
  - ✓ 不满足约束的候选方案一般应当在设计构思过程中被排除
  - ✓ 例如，“不能明文传输和保存密码信息”是一个绝对的约束，而“项目在2021年10月底交付并上线运行”则有一定弹性

功能和非功能性需求的实现

- 实现功能性需求的方式相对直观，即将所要求的功能性需求无一遗漏地分配到某个软件设计单元（例如组件、模块、类等）上即可
- 实现非功能性的质量需求则要复杂得多
  - ✓ 不仅取决于软件系统的组件如何划分而且还取决于一系列复杂的技术选型决策以及组件间交互设计
  - ✓ 例如使用什么样的数据库、使用什么编程语言和开发框架、组件之间采用何种通信协议和交互模式（如同步或异步）、组件如何部署以及实现运行时伸缩等

可**维护**性（Maintainability）
可**扩**展性（Extendibility）
性能（Performance）
可用性（Availability）
可靠性（Reliability）
信息安全性（Security）
防危安全性（Safety）
韧性（Resilience）

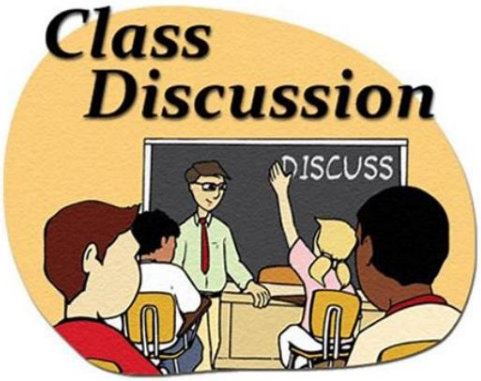
针对非功能性质量的体系结构设计考虑

质量属性	含义	体系结构设计考虑
可维护性 可扩展性	当产生新需求、发生需求变更或发现缺陷时，能够花费较少的时间和成本完成修改任务，需要修改的地方较少且影响范围可控	软件系统的组件粒度较小，各个组件相对独立，组件间松耦合；组件间依赖通过契约化的方式进行显式的声明，避免共享数据等隐藏的依赖和相互影响；通过可插拔组件等方式实现可灵活配置和扩展的可变点
性能	系统能支持大量的并发访问并保证响应时间在可接受范围内	软件系统或其组件能够以分布式的方式部署多个副本并实现负载均衡；副本数量能够根据需要进行伸缩（即副本数量增加或减少）；关键性操作集中实现和部署以避免频繁的分布式通信；使用缓存等手段提高数据访问性能
可用性 可靠性	系统能够随时提供自身的服务（即服务随时可用），这要求系统运行时很少失效或者失效后能尽快恢复	软件系统或其组件具有冗余副本，在失效时可以进行自动更新或替换；隔离不同组件及其副本，避免失效造成更大范围影响；使用超时、熔断、限流等机制及异步通信降低失效影响
信息安全性	系统中的信息不会被非法访问或篡改，功能不会在非授权情况下被使用，在遭遇恶意攻击或入侵时仍能保持一定的服务能力	对于系统功能和数据的访问进行充分的用户身份认证和权限检查；敏感信息进行加密传输和存储；系统对各种访问行为进行完整的日志记录和严密的审计；将关键性数据和其他资产置于层次化结构的最内层进行安全确认和保护
防危安全性	系统不会因为失效而对人身财产安全或环境造成危害或破坏	将可能造成危害或破坏的操作集中在一起实现并加以特别的保护；对于潜在的危害或破坏进行检测和预测，一旦发现问题及时停止相关操作以避免造成损失
韧性	系统在设备（如硬盘）失效或遭受网络攻击等不利情况下仍然能够持续提供系统的关键性服务	系统能够持续检测设备失效和网络攻击，并在检测到问题后进行防御和恢复；系统能够通过资源调配、服务降级等手段实现自适应调整，从而在不利情况下确保关键性服务持续可用

非功能性质量需求的冲突与权衡

- 不同的非功能性质量需求之间经常存在冲突，即实现某一个非功能性质量需求的设计可能会对另一个造成负面影响，例如
  - ✓ 面向可维护性和可扩展性所采取的细粒度组件以及灵活的可变性设计经常会对性能造成负面影响
  - ✓ 面向信息安全和防危安全采取的额外的验证、加密和保护措施对性能可能会有负面影响并带来额外的成本开销
- 软件体系结构设计经常需要在不同的非功能性质量需求之间进行权衡取舍，例如
  - ✓ 一个性能压力并不是太高但需要灵活适应不同客户需要的软件产品可以更突出对于可维护性和可扩展性的追求
  - ✓ 反之，一个对于性能要求十分苛刻的软件系统则有可能需要牺牲一部分可维护性和可扩展性

课堂讨论：非功能性质量需求的设计考虑



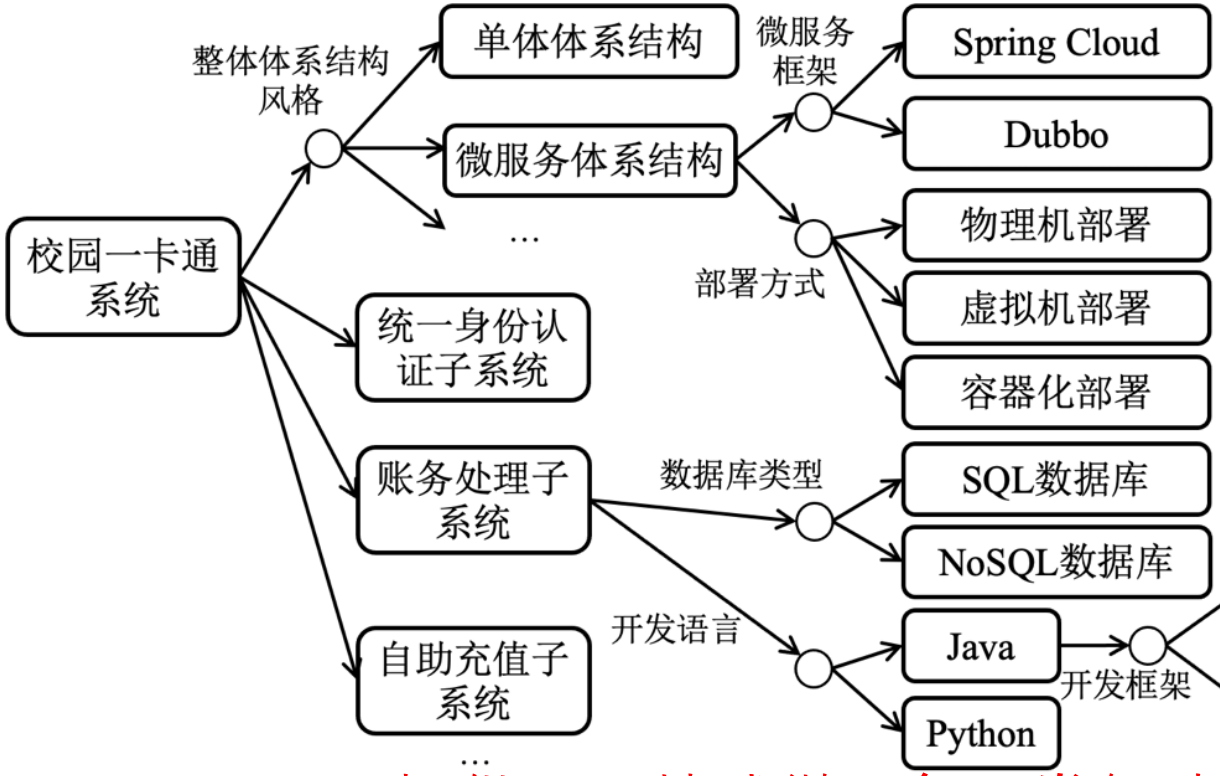
课堂讨论：一个火车票订票系统如果有以下非功能性质量需求，那么要考虑哪些设计上的对策?

支持百万级的用户并发访问
系统每月宕机不超过1次
系统每天用户不可用的时间不超过1分钟
系统不易被黑客入侵
系统可以方便增加各种增值服务

校园一卡通系统的部分体系结构设计决策

单体体系结构容易实现

微服务体系结构服务独立性和隔离性以及可扩展性和可用性更好



由于采用了微服务体系结构而且希望实现服务独立发布、部署和伸缩，因此容器化部署更合适

SQL数据库采用结构化存储，优势在于支持复杂查询、技术成熟

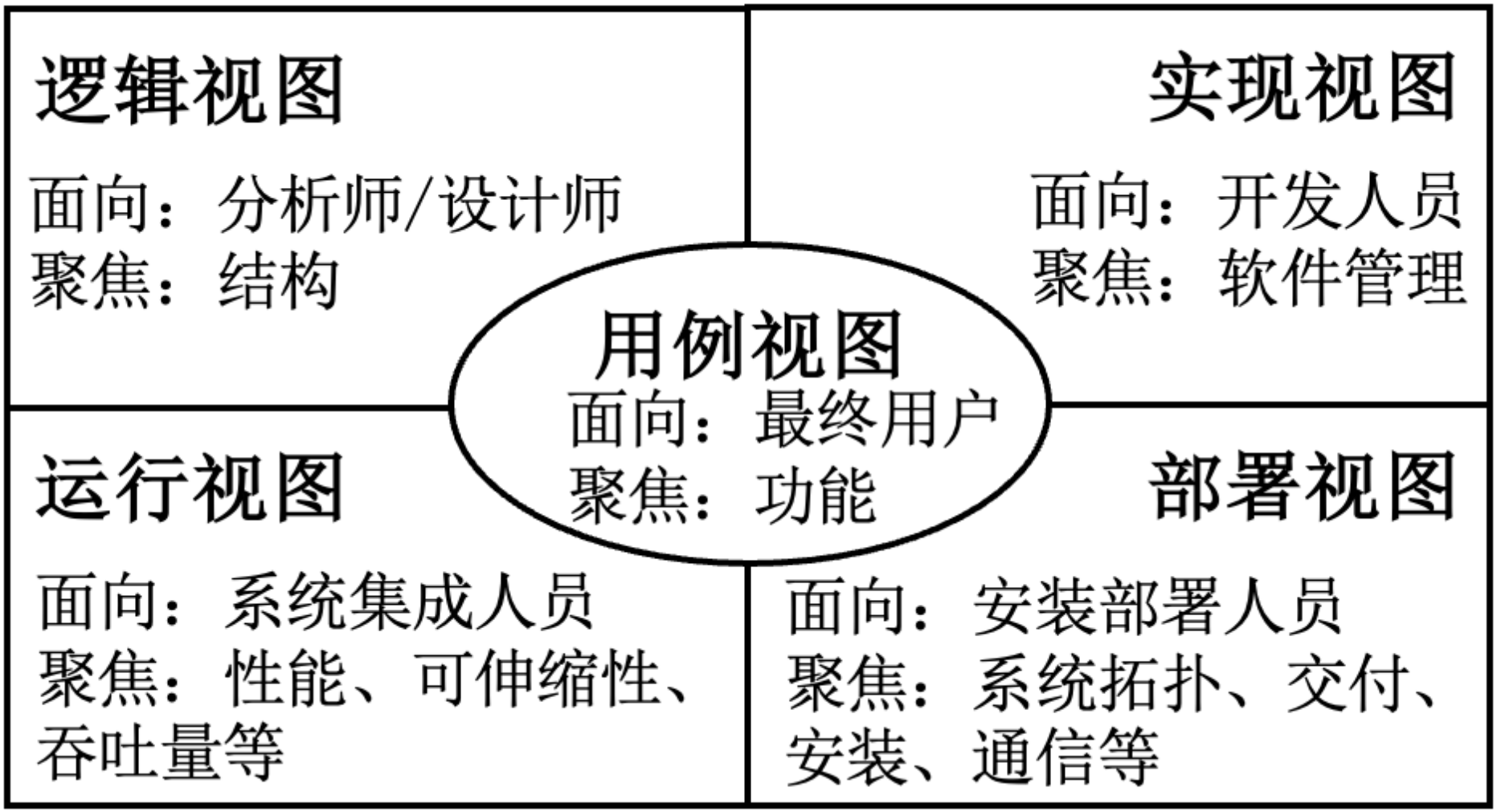
Spring Cloud提供了一站式微服务开发解决方案，组件齐全；源于Spring，天然支持Spring boot，便于业务快速落地

Dubbo则提供了一些服务治理的关键组件；支持基于RPC的服务间交互，因此性能较高，常用于电商系统

软件体系结构描述

- 软件体系结构设计需要通过一种规范和具体的方式进行描述，成为相关涉众可以共享访问的一种重要的软件开发文档
- 软件体系结构设计是一种高度抽象的复杂设计方案，难以通过单一的方式完整表达
  - ✓ 就像一栋建筑的设计方案难以通过一张图纸来表达，而是要从不同角度进行观察和投影，从而得到俯视图（鸟瞰图）、平面图、立面图、剖面图等一系列设计图纸
  - ✓ 软件体系结构设计也需要通过一种多视图的方式进行描述和理解，体现了不同涉众的不同关注点，例如：组件划分以及组件间接口定义、每个组件的开发要求、整个系统各个部分编译和构建的成分及其形态、各软件组件应当如何部署到不同的硬件设备上以及如何进行网络通信

软件体系结构的“4+1”视图



“4+1”视图的具体内容-1

- 用例视图（Use-Case View）：又称场景视图
  - ✓ 对软件系统与外部参与者之间的典型交换序列的场景化描述，从最终用户的角度刻画了系统需要实现的功能
  - ✓ 位于各体系结构视图的核心，是联系各种不同视图的纽带
  - ✓ 包括各种关键的用例和场景，以免体系结构设计遗漏重要的功能需求
- 逻辑视图（Logical View）
  - ✓ 反映软件分析和设计人员的关注点，即软件的业务模型以及逻辑分解结构
  - ✓ 可以描述目标软件系统的领域模型（概念、实体及其之间的关系）、关键数据模型与数据流
  - ✓ 还可以描述系统的高层组件（例如单体体系结构中的模块、微服务架构中的服务以及所复用的自有或第三方组件等）及其之间的关系

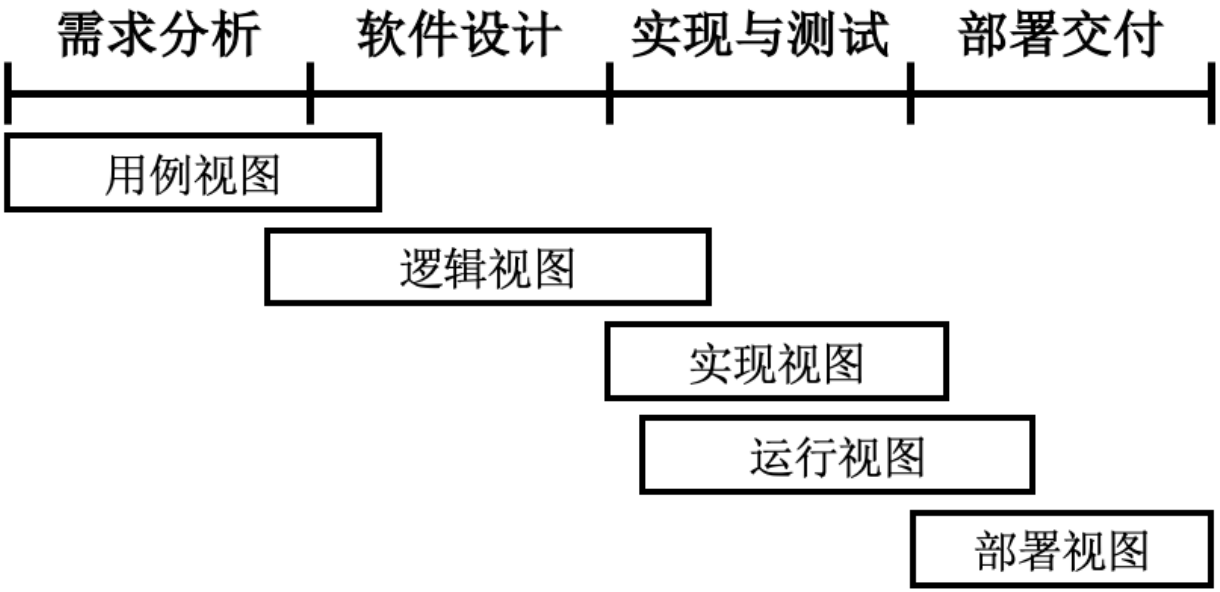
“4+1”视图的具体内容-2

- 实现视图（Implementation View），又称开发视图
  - ✓ 反映软件开发者以及开发管理者的关注点，即系统是如何分解实现以及构建得到最终交付制品（如二进制发布包）
  - ✓ 描述代码的组织结构，即代码仓库以及代码目录结构
  - ✓ 还可以描述构建（build）模型，即各个组件如何与所依赖的外部组件一起通过编译构建得到交付制品
- 运行视图（Process View）
  - ✓ 反映系统集成人员的关注点，即组件之间的动态交互行为以及其中的并发和同步设计
  - ✓ 可以描述系统的进程划分和定义、组件等设计元素如何分布在不同的进程之中、进程之间如何进行交互和同步
  - ✓ 对于分析系统的性能、并发性、分布性、可伸缩性、吞吐量、容错能力等质量属性具有重要的作用

“4+1”视图的具体内容-3

- 部署视图（Deployment View），又称物理视图
  - ✓ 反映安装部署人员的关注点，即软件系统如何交付并部署到运行环境中
  - ✓ 可以描述交付和部署包的结构，即在构建得到的交付制品基础上如何进行组织和打包从而得到可以在运行环境中部署的软件包
  - ✓ 还可以描述面向运行环境的软件部署方案，包括硬件节点类型、每类节点的设备要求、设备之间的网络连接方式、设备上的运行环境及参数配置、交付和部署包在硬件节点上的部署位置等

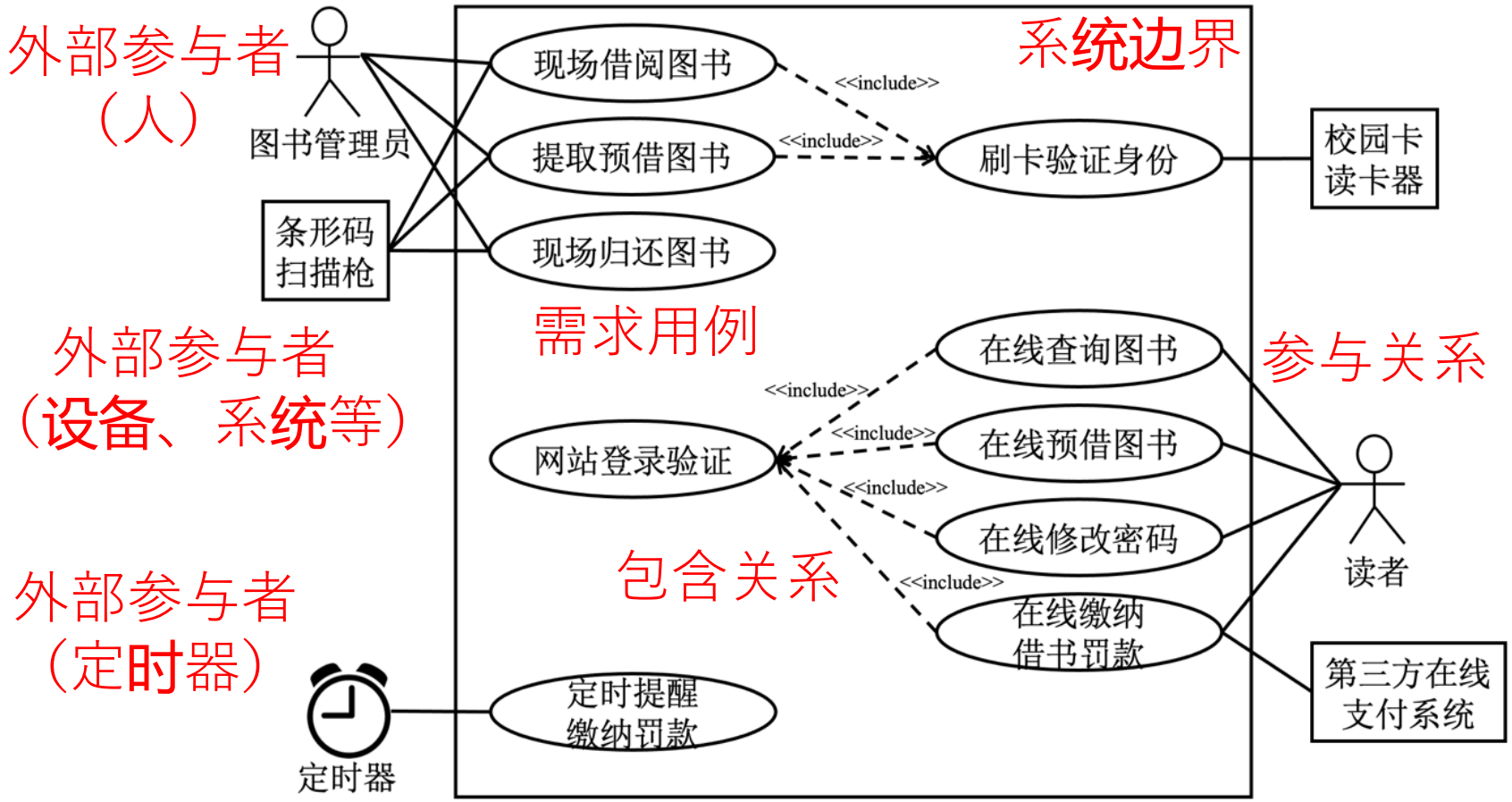
软件体系结构视图的作用阶段



- 用例视图和逻辑视图主要指导需求分析和软件设计过程
- 逻辑视图还会为实现与测试提供软件高层逻辑结构方面的指导
- 实现视图指导实现与测试阶段的任务分工以及部分集成和构建活动，同时面向部署交付的需要定义交付制品
- 运行视图指导实现与测试阶段的进程划分与定义、进程间交互的集成与测试等，同时面向部署交付提供进程及交互关系定义
- 部署视图指导部署交付阶段的部署交付包的组织以及面向运行环境的软件部署方案

校园一卡通系统体系结构示例：用例视图

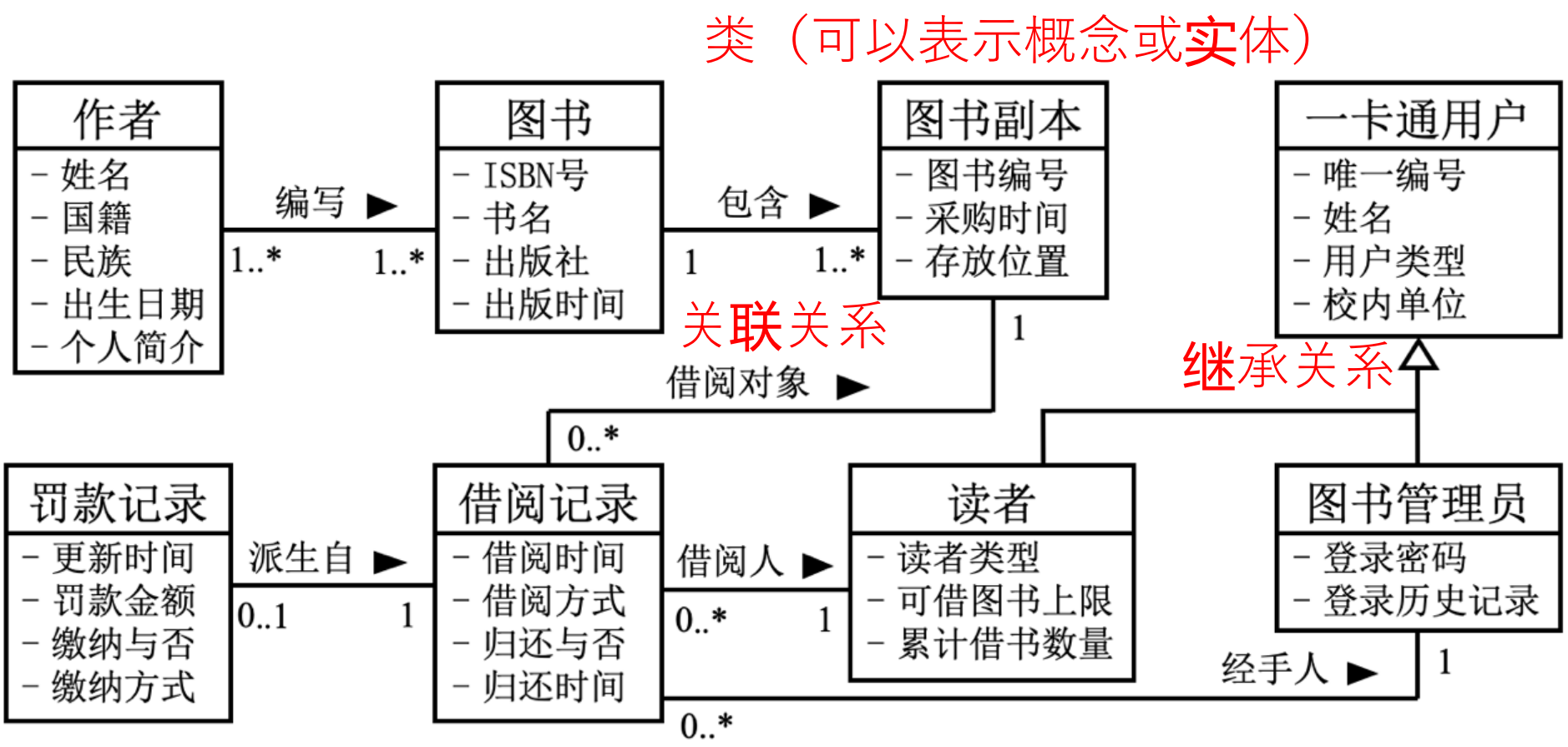
使用UML用例图（Use Case Diagram），描述系统与外部参与者之间的交互场景，其中每一个用例表示一类交互场景



还可以使用UML顺序图（Sequence Diagram）或活动图（Activity Diagram）描述每一个用例内部的具体交互过程

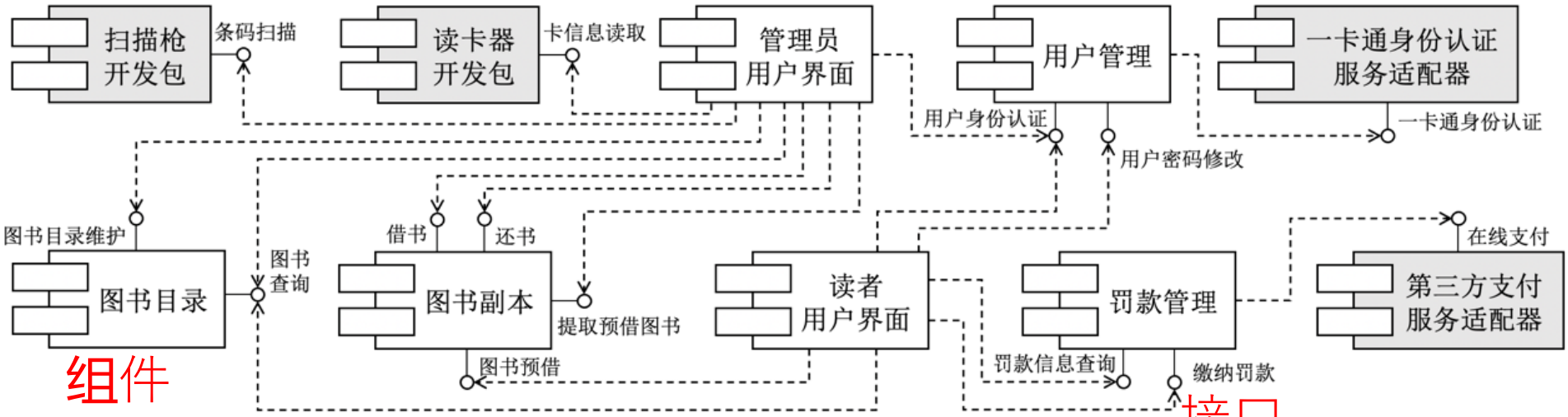
校园一卡通系统体系结构示例：逻辑视图

领域模型和数据模型可以使用UML类图（Class Diagram）来描述



校园一卡通系统体系结构示例：逻辑视图

组件结构可以使用UML组件图（Component Diagram）来描述

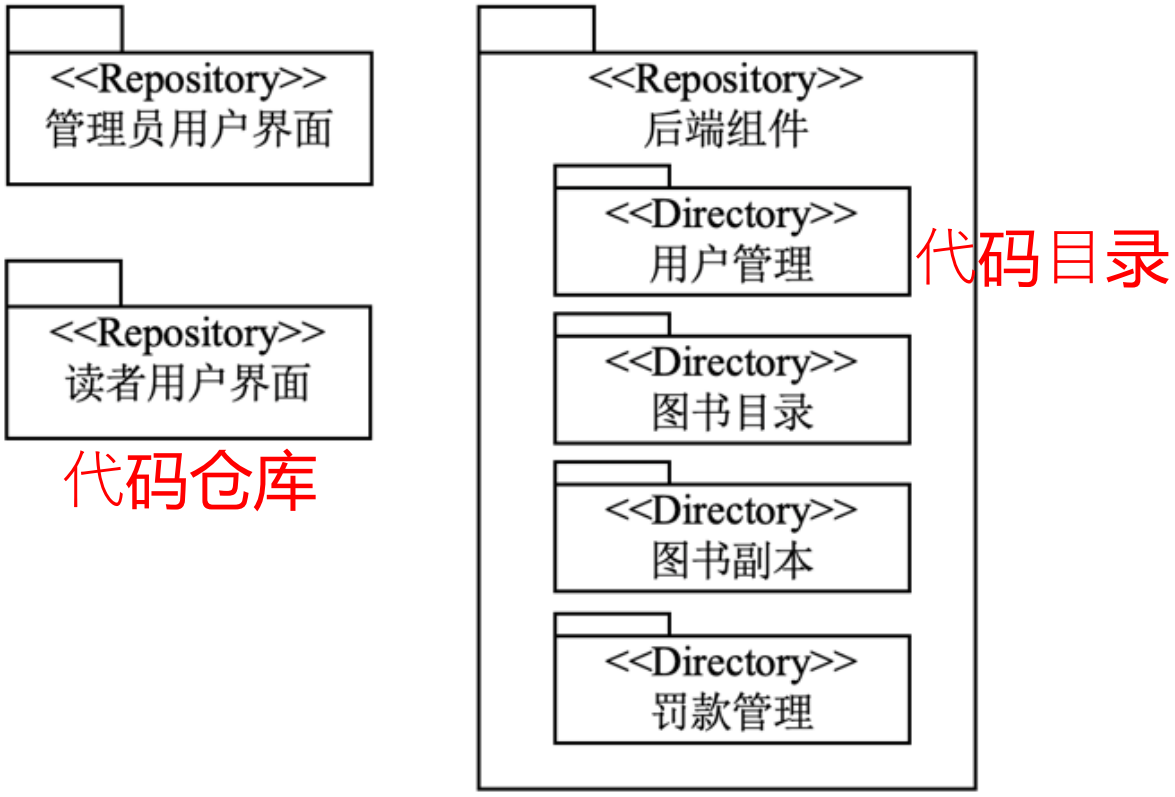


灰底的组件表示由第三方提供的开发包、适配器等外部组件

逻辑视图中的组件是一种高层的系统组成元素，其本身还可以由多个子组件或文件来实现

校园一卡通系统体系结构示例：实现视图

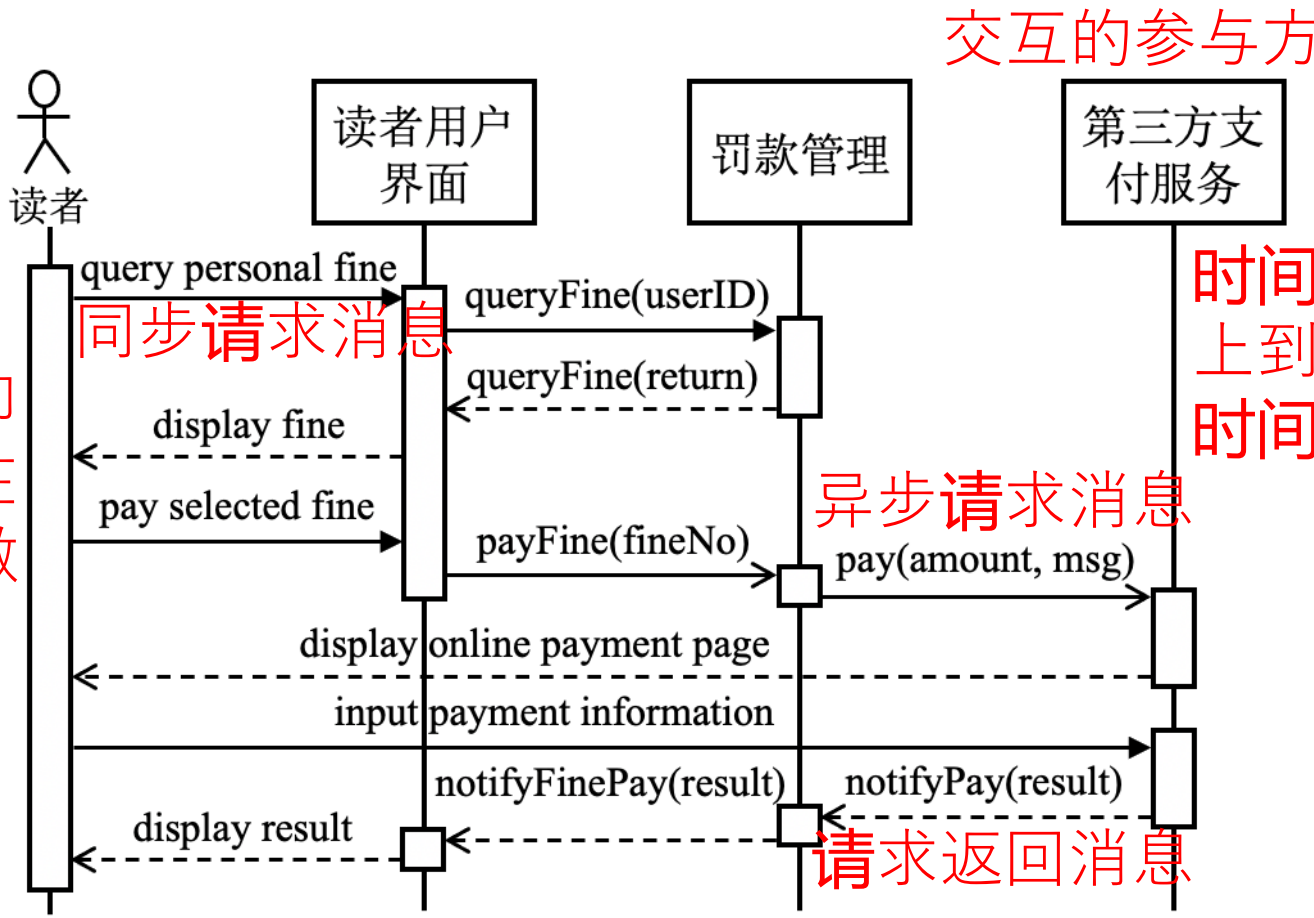
代码组织结构可以使用UML包图（Package Diagram）来描述



每一个矩形组合代表一个包，这些包通过使用UML中的“构造型”（Stereotype）这一机制被扩展定义成了代码仓库（Repository）或者代码目录（Directory）

校园一卡通系统体系结构示例：运行视图

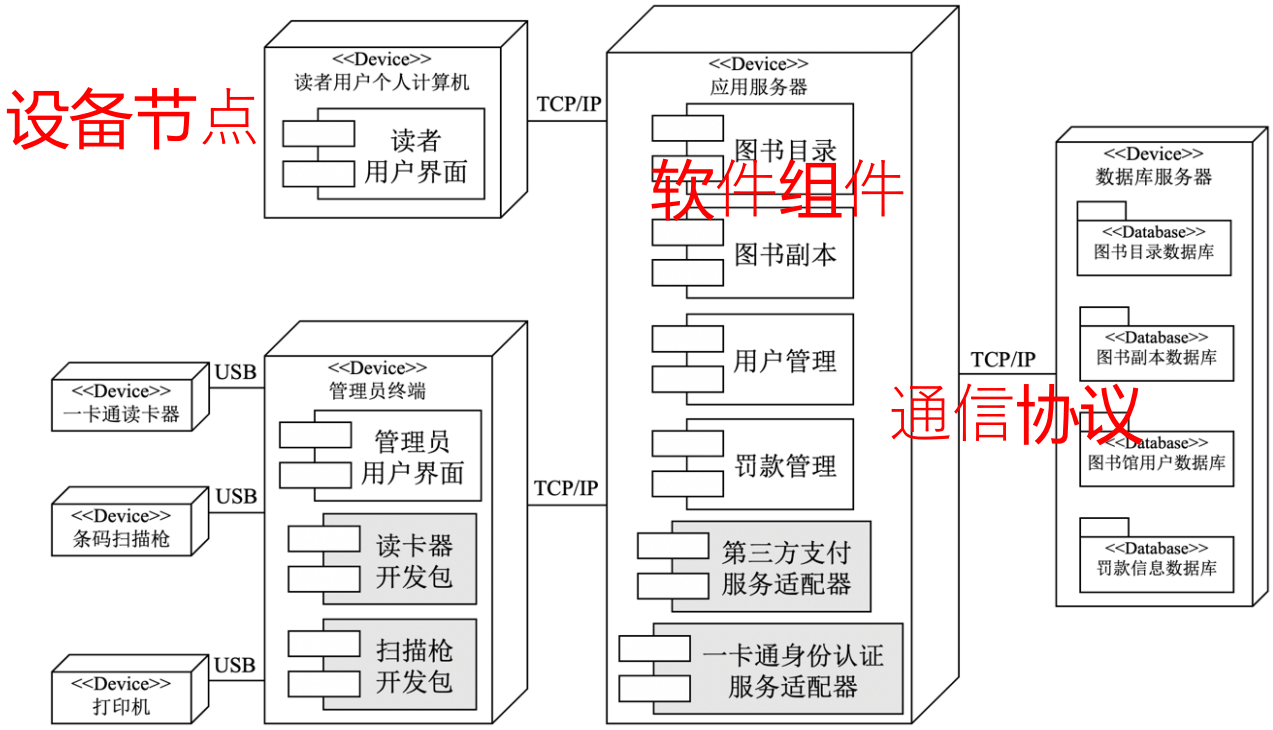
进程交互关系可以使用UML顺序图（Sequence Diagram）来描述



运行视图可以帮助我们分析进程之间的交互关系，从而对系统性能、可靠性等方面的质量属性进行推断

校园一卡通系统体系结构示例：实现视图

面向运行环境的软件部署方案可以使用UML部署图（Deployment Diagram）来描述



每一立方体通过使用UML中的“构造型”（Stereotype）这一机制被扩展定义为设备节点（Device）

如果需要，部署视图还可以描述所部署的组件对应的软件包是如何在构建得到的交付制品基础上打包产生的

建筑风格




哥特式建筑是11世纪下半叶起源于法国，13～15世纪流行于欧洲的一种建筑风格。主要见于天主教堂，也影响到世俗建筑。

哥特式建筑最明显的建筑风格就是高耸入云的尖顶及窗户上巨大斑斓的玻璃画，特色包括尖形拱门、肋状拱顶与飞拱。

最富盛名的哥特式建筑有俄罗斯圣母大教堂、意大利米兰大教堂、德国科隆大教堂、英国威斯敏斯特大教堂、法国巴黎圣母院等。

https://baike.baidu.com/item/哥特式建筑




中国古典园林艺术是指以江南私家园林和北方皇家园林为代表的中国山水园林形式。

所有建筑，其形与神都与天空大地等自然环境吻合，同时又使园内各部分相接，以使园林体现自然，淡泊，恬静，含蓄的艺术特色，并收到移步换景，渐入佳境，小中见大等观赏效果。师法自然，融于自然，顺应自然，表现自然。

中国现存的古典园林有颐和园、承德避暑山庄、拙政园、沧浪园等。

https://baike.baidu.com/item/中国古典园林

软件体系结构风格

- 以一种抽象的方式刻画了软件系统的整体设计样式，包括系统中的组件类型以及不同类型的组件之间的关系和组织方式等
- 每一种风格也都有其明确的适用条件和优缺点

C/S（Client/Server）风格

- ✓ 功能强大并针对特定操作系统专门开发的客户端软件
- ✓ 大部分业务逻辑可以在客户端实现，而服务器端可以相对简单甚至只提供数据库服务
- ✓ 优点：可以实现复杂的客户端逻辑并充分利用客户端计算机资源

B/S（Browser/Server）风格

- ✓ 依赖于用户计算机上所安装的浏览器来展现界面并实现用户交互
- ✓ 大部分业务逻辑在服务器端实现
- ✓ 优点：无需安装客户端、升级方便

常见的软件体系结构风格

- 经典的软件体系结构风格
  - ✓ 层次化风格
  - ✓ 黑板风格
  - ✓ 管道和过滤器风格
- 其他相关风格
  - ✓ 调用和返回风格
  - ✓ 面向对象风格
- 新的适应云的特点的微服务体系结构风格
- 不同的软件体系结构风格之间并不互斥，可以在同一个软件系统中结合使用
  - ✓ 例如，一个采用B/S体系结构风格的软件系统可以同时采用层次化体系结构风格，并在其中的业务层采用面向对象体系结构风格

层次化体系结构风格



- ✓ 每个层次都有明确的功能和职责划分，上层依赖于下层而下层则不能依赖于上层
- ✓ 通过明确定义的层间接口隔离不同层次内部实现方式的影响
- ✓ 体现了分解和抽象的设计原则：通过层次划分将功能实现分解到不同层次上；通过层间接口对每个层次的内部实现进行抽象

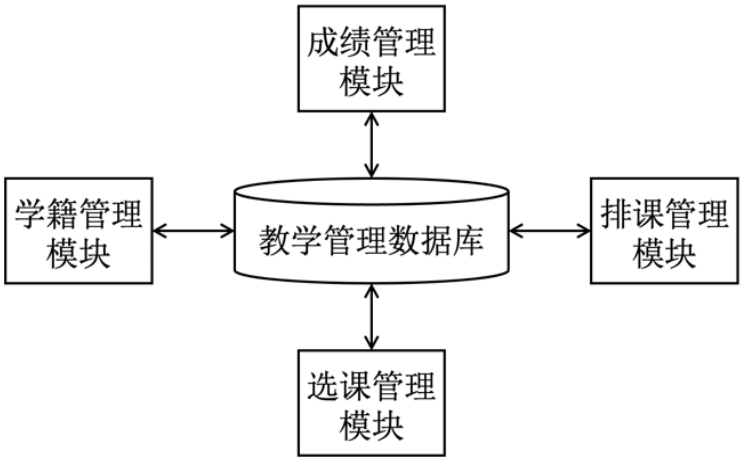
优点

- ✓ 清晰的层间接口定义和抽象有利于将开发任务解耦
- ✓ 清晰的层次划分和层间接口定义隔离了变更的影响
- ✓ 标准化的层间接口使得开发人员可以方便地替换其中的某一层，提高了可移植性

缺点

- ✓ 清晰的层次结构定义有时候较为困难
- ✓ 过多的分层可能对系统性能造成负面影响
- ✓ 高层实现不能直接访问底层资源，从而无法针对性地进行定制和优化

知识库体系结构风格



- ✓ 一种以数据为中心的体系结构风格
- ✓ 以数据的形式存在的知识在中心知识库中被集中保存和管理，围绕知识库的一组软件组件通过共享知识库间接进行交互
- ✓ 又被称为黑板风格，用于解决求解过程未知的人工智能问题

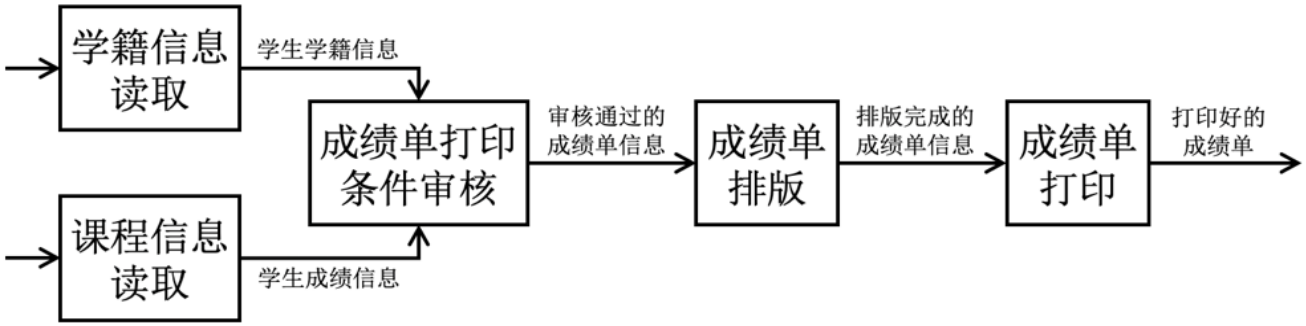
优点

- ✓ 实现简单并且能够依托成熟的数据库系统实现大量数据的共享
- ✓ 围绕中心知识库的各个组件之间耦合度很松，可以方便地实现组件的增加和修改而不会带来太大的副作用

缺点

- ✓ 当中心知识库数据量大到一定程度之后难以实现高效的数据访问和管理
- ✓ 中心知识库中的数据格式和规范对于开发有很大影响，一旦定义不合适可能会导致相关组件难以扩展和修改

管道—过滤器体系结构风格



- ✓ 一种数据流体系结构，即其中的组件是按照数据流进行组织
- ✓ “过滤器”表示实现数据加工和处理的软件组件，而“管道”则表示这些组件之间的连接关系并掌握数据流的走向
- ✓ 组件间关系主要体现在输入输出数据流上，因此耦合相对较松

优点

- ✓ 适合于以数据传输和处理逻辑为主的系统（如数据批处理系统）
- ✓ 这种系统中包含一系列数据处理组件而且几乎不包含任何与用户交互的内容

缺点

- ✓ 人机交互性较强的软件系统，例如需要图形用户界面支持的系统，一般不适合采用管道—过滤器体系结构风格

分布式软件体系结构

- 分布式系统（Distributed System）
  - ✓ 软件系统中包含多个通过网络连接的计算设备（称为节点）
  - ✓ 其中的软件分布在不同的计算设备上并通过网络进行通信
- 分布式系统广泛应用的原因
  - ✓ 移动终端（如智能手机）、传感器（如温度传感器）、智能设备（如智能冰箱）等网络化设备的大量使用
  - ✓ 软件应用的用户数和使用频次越来越高，使得软件应用必须使用多台服务器甚至云计算平台来进行分布式部署
- 分布式软件体系结构设计具有更强的挑战性
  - ✓ 需要考虑软件组件如何分布在不同的设备上并实现高效的分布式通信，同时还要确保一致性
  - ✓ 例如，网上购物系统需要在多台服务器上部署与订单处理相关的软件组件以提高并行处理能力，同时也要设计相应的同步机制以确保一致性（如成功支付的订单一定送货）

分布式软件系统的质量要求

- 高性能（Performance）：在高并发访问的情况下能够实现较高的吞吐量（即单位时间内成功处理的请求数）并确保较低的延迟（即从客户端发起请求到收到响应的时间间隔）
- 高可用性（Availability）：在任意时间访问时系统都可以以较大的概率正常运行并提供所请求的服务，这要求系统有较好的容错性（即使某些组件失效也能继续提供全部或一部分服务）以及较强的故障检测和恢复能力
- 可伸缩性（Scalability）：系统具有良好的弹性，即系统的处理能力（包括相应的资源开销）能够随着负载的上升而随之扩张以及随着负载的下降而随之收缩
- 高安全性（Security）：系统难以被外部攻击或利用从而导致非法访问、信息泄漏、数据篡改或系统不可用

分布式软件体系结构设计除了满足功能性要求外，一般还需要考虑以上几个方面的质量要求

分布式软件体系结构设计的常见问题

设想，一个分布式系统为了提高并发处理能力经常需要在多个服务器节点上部署相同或相关的组件，其中可能包含一些相同的数据内容，例如多个支持负载均衡的商品查询服务器上都保存着同样的一些商品数据（如编号、名称、数量等）。此时，如果某一个客户端请求需要更新这些内容，例如商品购买成功后需要更新对应商品的数量，那么相关服务器上的对应商品数量都需要更新。

- 为了确保不同服务器上商品数量的一致性，更新操作需要同时发送给相关的服务器，但如果其中有一些服务器或其网络发生故障而导致更新失败，那么就会导致更新成功和更新失败的服务器之间发生商品数量不一致的问题
- 如果要严格确保一致性，那么就要求相关服务器上的数据更新全部都成功后系统才能正常提供服务，由此可能导致系统的整体可用性下降（考虑各种服务器或网络故障问题）

软件体系结构设计经常要在多个相互冲突的目标之间进行权衡决策

分布式软件体系结构设计原则

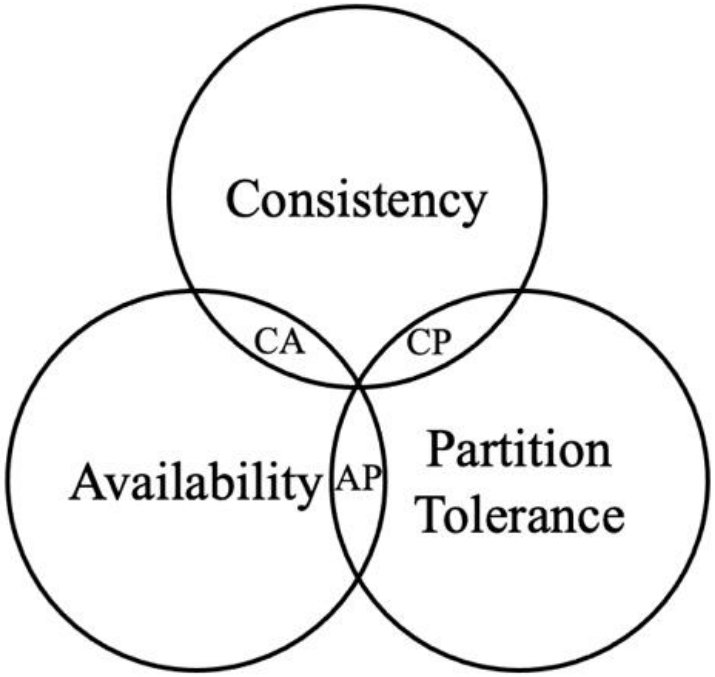
- 我们需要明确分布式软件体系结构设计的相关原则，从而指导我们做出合理的权衡决策
- 以上所讨论的问题可以用CAP定理（CAP Theorem）来说明
  - ✓ 最初源自于加州大学伯克利分校的计算机科学家Eric Brewer在2000年的分布式计算原理会议上所作的题为“Towards Robust Distributed Systems”的主题演讲中的一个猜想，因此又被称作Brewer定理（Brewer’s Theorem）
  - ✓ 2002年，来自麻省理工学院的Nancy Lynch和Seth Gilbert证明了这一猜想（Lynch et al.，2002），使之成为一个定理

CAP定理

对于同时存在数据读取（如读取商品数量）和更新（如更新商品数量）操作的分布式系统，一致性（Consistency）、可用性（Availability）、分区容错性（Partition Tolerance）三个属性无法同时被满足，即如果想满足其中两个那么剩下的一个必须被牺牲

一致性：数据更新后任何一个客户端在任何时候访问都能读取到最新的数据，就好像这些数据只有一份统一的拷贝放在一个节点上

可用性：向一个非故障状态的节点发出的请求总是可以在合理的时间内返回正确的响应（但所返回的数据可以不是最新的）



分区容错性：系统中某一个节点或网络分区发生故障后，系统仍然能够对外正常提供服务，这意味着系统的多个网络分区中都部署了数据和服务副本

CAP定理的权衡取舍

- 三种权衡取舍
  - ✓ 实现一致性和可用性（CA），此时放弃分区容错性，即采用单节点部署数据和服务
  - ✓ 实现一致性和分区容错性（CP），此时放弃可用性，即要求在多副本数据完成同步更新之前停止进行请求响应，容忍由此带来的可用性问题
  - ✓ 实现可用性和分区容错性（AP），此时放弃一致性，即允许不同分区返回不一致的结果
- 互联网应用系统（如在线购物）的权衡考虑
  - ✓ 用户无法接受系统时不时就无法访问或者某些商品暂时无法查询或购买，因此可用性是必须满足的
  - ✓ 这些系统都需要使用大量的服务器和网络分区并部署多个服务和数据副本，从而使分区容错性成为一种必要的选择
  - ✓ 是否可以放松一致性的要求就成为一个很多人思考的问题

从CAP定理到BASE原则

- 在大量的互联网应用软件开发实践基础上，人们逐渐形成了更具实践指导意义的BASE原则
- BASE原则最初由eBay的架构师Dan Pritchett提出，其主要思想是牺牲数据的强一致性来换取高可用性并在最后能实现最终一致性
  - ✓ 基本可用：不追求绝对的系统可用性，而是允许在服务质量有所下降（服务降级）的情况下确保基本的可用性
  - ✓ 软状态：又称弱状态，即允许系统中不同节点上的数据处于一种不一致的中间状态，这意味着不同节点上的数据更新可以存在一定的时间差
  - ✓ 最终一致性：系统中不同节点上的数据副本经过软状态的过渡后最终都能够达到一致的状态

BASE原则带来的设计灵活性

- BASE原则和最终一致性的思想使得分布式软件体系结构设计具有了更大的灵活性
  - ✓ 由于放松了强一致性要求，系统可以通过多副本部署提高系统的处理能力以及应对局部服务器或网络失效的能力
  - ✓ 同时根据负载的变化灵活调整副本的数量，从而实现较高的性能、可用性和可伸缩性
- 架构师可以根据实际情况做出不同的权衡决策

如果一个在线购物系统部署了多个商品信息查询和购买服务及其数据的副本并确保可用性。由于要确保可用性，因此这些副本可能在未完成数据同步的情况下分别响应了不同用户的请求，从而导致数据不一致（例如某件商品在剩余数量不足的情况下同时卖给了多个用户）。这种不一致对于在线购物系统而言可能是可接受的，因为系统可以在后续配送环节想办法调货满足超卖的订单或者退回所支付的款项和抵扣的积分，从而实现最终一致性。

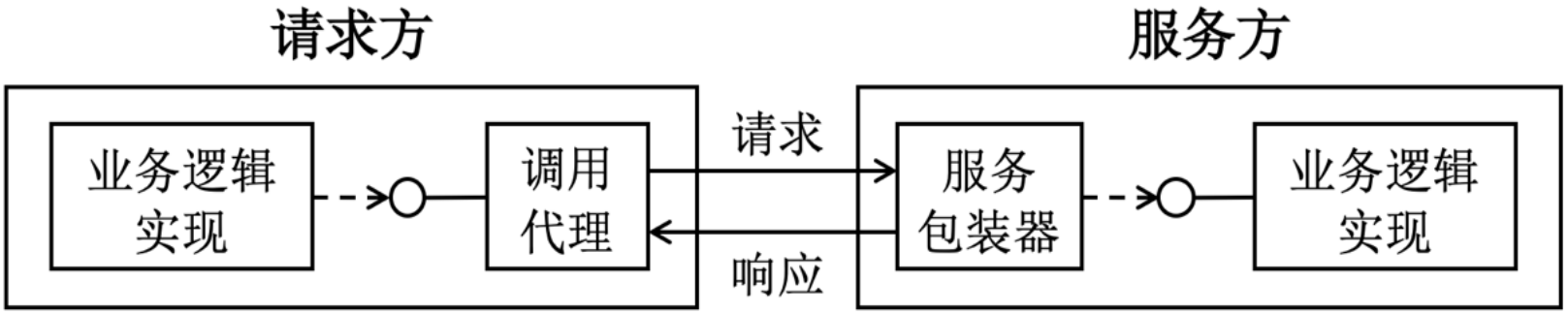
进程间通信

- 与本地调用相比，进程间通信更加复杂而且需要考虑通信延迟和可能的调用失败等问题
- 在分布式软件体系结构设计中，组件间交互技术的选择是一个重要的技术决策
  - ✓ 一对一同步模式：客户端发出请求后进行阻塞等待，而服务端会有一个唯一的实例处理该请求并实时响应
  - ✓ 一对一异步模式：客户端发出请求后无需阻塞而是继续执行，服务端会有一个唯一的实例处理请求并进行响应（例如回调客户端接口）或不进行响应（此时是客户端对服务端的单向通知）
  - ✓ 一对多的异步模式：客户端以消息的方式发出请求后继续执行，多个服务实例通过消息订阅收到通知或者产生多个回调消息作

	一对一	一对多
同步模式	请求/响应	无
异步模式	异步请求/响应、单向通知	发布/订阅、发布/异步响应

远程调用

- 一对一的请求/响应式的组件交互一般通过远程调用的方式来实现
- 一般都会采用某种开发框架来将业务逻辑实现与远程调用及其所包含的进程间通信相解耦
  - ✓ 请求方（即客户端）会引入一个调用代理，这样其实现代码可以以本地调用的方式实现所需要的远程调用
  - ✓ 服务方（即服务端）则会引入一个包装器，将其业务逻辑实现包装成可以跨进程访问的服务
  - ✓ 二者被称为桩（stub）和骨架（skeleton），一般由远程调用框架来提供，因此开发者无需了解其中的进程间通信实现方式



RESTful接口

- 一种基于Web服务的软件体系结构风格
  - ✓ 由可独立部署的Web服务组件组成（一般是无状态的），因此需要调用服务的客户端保存状态并在服务调用时提供所需要的状态信息
  - ✓ 优势是灵活伸缩，因为不需要在增加实例时进行状态同步
- 符合REST风格（即RESTful）的Web接口
  - ✓ 以资源为核心，包括业务对象（或者业务对象的集合），使用URL来作为唯一标识符并进行引用
  - ✓ 通过HTTP动词来操作资源：GET、POST、PUT、DELETE
  - ✓ 通过资源URL与HTTP动词的组合来定义Web服务接口

GET / books/{bookID}：图书信息查询接口
POST / books：新增图书接口
PUT / books/{bookID}：更新图书信息接口
DELETE / books/{bookID}：删除图书接口

RESTful接口的优缺点

- 主要优点
  - ✓ 规范简单易用，熟悉的人多
  - ✓ 支持浏览器扩展（如Postman）或命令行直接测试
  - ✓ 直接支持请求/响应方式通信，无需使用中间代理
  - ✓ 使用的HTTP协议对防火墙友好（一般都能透过防火墙）
- 主要缺点
  - ✓ 只支持请求/响应方式的通信
  - ✓ 调用期间服务需保持在线，因此可能降低可用性
  - ✓ 单个请求中获取多个资源不方便
  - ✓ 难以将多个更新操作映射到有限的HTTP动词
  - ✓ 客户端必须知道服务实例的位置（URL），因此需要使用服务发现

RPC（远程过程调用）

- RPC通过二进制消息进行通信，因此性能更好、延迟更小
- 以操作为核心的，可以灵活定义各种接口操作
- 客户端组件（请求方）
  - ✓ 首先调用本地调用代理（即客户端桩）
  - ✓ 然后本地调用代理将请求信息（包括调用的接口及参数）进行序列化包装使之成为能够通过网络传输的二进制消息并将消息发送给服务端
- 服务端组件（服务方）
  - ✓ 服务包装器收到消息后会进行反序列化操作，即从二进制消息中解析出所调用的接口及其参数
  - ✓ 然后调用本地业务逻辑实现代码
  - ✓ 服务端完成处理后，返回消息以类似的方式发送给客户端

基于消息的通信

- 基于消息的异步通信：类比发送及回复短信
- 异步通信更加灵活，消息接收方可以灵活决定按照何种顺序以及何时处理消息，而发送方还可以将消息发给多个接收方
- 基于消息通信也可以实现类似于同步调用的效果，即用异步来实现同步：客户端发送请求消息后可以停下来反复查看响应消息有没有收到，收到后马上对响应消息进行处理（就像一个心急的人发完短信后不断查看对方有没有回消息而无心做其他事情）

基于消息的通信中的消息队列

- 基于消息的通信一般需要使用消息队列（Message Queue，简称MQ）来实现
  - ✓ 消息的发送方（消息的生产者）将消息放入消息队列中
  - ✓ 消息队列作为一种容器保存消息
  - ✓ 消息的接收方（消息的消费者）从消息队列那里获取消息并进行处理
- 消息的构成
  - ✓ 消息头（header）：包含消息ID、回复地址等元信息
  - ✓ 消息体（body）：消息正文的数据内容（文本或二进制）
- 消息内容的类型
  - ✓ 数据：业务数据内容，例如图书ISBN号、标题、作者等
  - ✓ 命令：类似远程调用请求，包含操作及参数，如预借图书
  - ✓ 事件：表明发送方处业务对象状态发生了变化，需要通知对此感兴趣的其他相关方，例如图书馆新到图书上架

负载均衡

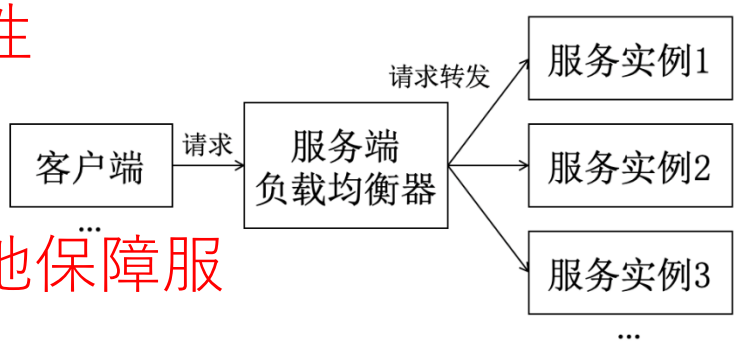
- 分布式软件系统并发请求量大到一定程度时就必须要依靠很多台服务器一起来分担请求负载
- 这些服务器可以提供一样的服务能力和内容
- 负载均衡：将大量的请求负载以一种合理、均衡的方式分配到不同的服务器上
- 一旦做的不好可能造成服务器负载不均，从而导致整体处理能力不足以及计算资源的浪费
- 负载均衡器以软件的方式实现并通过各种策略和算法实现负载均衡调度
  - ✓ 服务端负载均衡
  - ✓ 客户端负载均衡

服务端负载均衡

- 目前主流的负载均衡实现方式
  - ✓ 客户端发出的请求由服务端的负载均衡器接收
  - ✓ 负载均衡器再按照某种策略选择一个服务实例进行转发
- 一般通过反向代理的方式实现：处于外网的客户端发出请求后，服务端的负载均衡器选择一个内部的服务实例来处理此请求
- 实现方式
  - ✓ 软件负载均衡器：nginx（最常用）、LVS、HaProxy
  - ✓ 硬件负载均衡器：利用路由器等硬件实现，如F5、Radware

通过反向代理实现了隔离，增强了安全性

支持文本、图片等静态资源的缓存功能

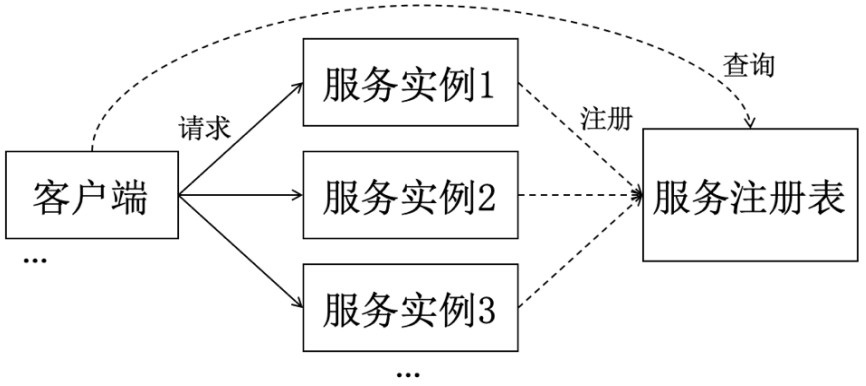


能够及时更新可用服务实例列表并更好地保障服务响应速度，同时客户端开发较为简单

客户端负载均衡

- 客户端发送请求时直接决定要访问的服务实例
  - ✓ 服务端维护一个服务注册表，每个服务实例创建时进行注册，并在后续过程中通过心跳机制维持联系
  - ✓ 客户端在本地保存一份服务实例列表，通过查询服务注册表进行定时更新，同时在发现不可用服务实例后将其移除
  - ✓ 客户端发起请求时先从本地列表中查询当前可用的服务实例，然后根据负载均衡策略选择其中一个来发送请求
- 实现方式： Spring Cloud Ribbon
  - ✓ 一个常用的客户端负载均衡插件，可以以软件库的形式集成到客户端中，并与Eureka 等服务注册与发现服务相结合使用

避免负载均衡器单点故障的影响，因为每个客户端本地保存了服务列表



常用的负载均衡策略

- 随机选择：随机从可用服务实例列表中选择一个进行请求
- 轮询（加权轮询）：将请求轮流分配给各服务实例，可以根据处理能力为服务实例加上权重
- 最小负载（加权最小负载）：根据各服务实例当前的请求负载（例如等待队列的长度）分配请求，可以根据处理能力为服务实例加上权重
- 最快响应速度：根据各服务实例的请求响应速度分配请求，但要排除服务实例在失效后快速返回的情况（即很快响应但实际上是失败）

分布式存储

- 分布式软件系统中高效的数据存取经常是一个关键问题
- 增加服务器和服务实例后，很多服务实例都需要访问持久化的数据存储（数据库或文件），从而使数据存储和访问成为瓶颈
- 为了缓解数据访问瓶颈，分布式软件系统中经常需要采用分布式的数据存储方案
- 相关技术和系统
  - ✓ 分布式文件系统
  - ✓ 分布式数据库
  - ✓ 分布式缓存

分布式文件系统

- 部署在由多台服务器组成的集群上，一般采用主从（Master/Slave）结构进行组织
  - ✓ 一个主节点管理整个文件系统的命名空间以及客户端对文件的访问操作
  - ✓ 多个从节点则管理所存储的数据（因此又称数据节点）
  - ✓ 主节点管理数据节点并向它们转发文件存取访问请求，如果一个数据节点无法正常工作则会被暂时移除
  - ✓ 每个数据块都会在多个数据节点上进行存储以实现冗余备份，主节点根据可用副本情况决定数据节点间的数据复制
- 对外提供统一的文件访问（例如创建、读取和更新文件），客户端无需了解内部结构
- 典型代表是作为Hadoop 一部分的HDFS（Hadoop Distributed File System）

分布式数据库

- 一种可以部署在网络上的多台服务器上，物理上分散但逻辑上统一的数据库
- 客户端访问分布式数据库时可以将其作为一个整体发出访问请求（例如数据库查询），而无需关心其背后的分布式结构
- 一般也采用主从结构进行组织
- 常用的分布式数据库
  - ✓ 一般都是非关系型的数据库（NoSQL）
  - ✓ Hbase（基于Hadoop HDFS）以及Apache Cassandra是一种基于键/值存储的列式数据库（即数据库表按照列而不是行来存储数据）
  - ✓ MongoDB是一种支持类似JSON/BSON格式的文档数据库，可以存储比较复杂的数据类型

分布式缓存

- 缓存：将经常被读取且较少变化的数据放到内存中，这样应用服务读取这些数据时无需访问数据库而只需要访问内存即可
- 分布式缓存类似于分布式数据库，可部署在多台服务器上并支持访问请求的转发和数据复制
  - ✓ 应用服务自行决定相应的缓存策略，例如读取哪些数据时需要尝试访问缓存、何时以及如何进行缓存数据更新等
  - ✓ 缓存数据一般会设置有效期，超过有效期未更新的数据就会失效（缓存系统可能会清理失效数据）
  - ✓ 如果某个缓存数据过期同时又有大量访问请求，那么将导致大量的数据库访问和数据库服务器宕机（称为缓存击穿）
- 使用最广泛的分布式缓存系统之一是Redis

可靠性保障

- 分布式软件系统面临外部不可预知的访问负载以及内部网络和服务器的各种不确定问题，如何确保其可靠运行是一个难题
- 体系结构层面上的可靠性保障策略建立在软件工程所蕴含的综合权衡的思想基础上
  - ✓ 当系统无法保证完全正常运行时可以通过降低服务质量、牺牲一些相对不那么重要的功能和服务、甚至拒绝一部分访问请求等手段确保系统维持基本运转
  - ✓ 相关的可靠性保障策略包括限流、降级、熔断等

可靠性保障策略

- 限流：对于超过流量限制的请求拒绝访问或者转入一个队列进行等待
- 降级：通过适当降低服务质量和用户体验来确保核心服务的可用性
  - ✓ 确保重要性较高的服务能够优先获得所需要的资源
  - ✓ 常用的降级策略：停用优先级较低的服务、降低服务质量、降低信息或服务的时效性
- 熔断：一种过载保护机制（类似于电路跳闸）
  - ✓ 上游服务断开对下游服务的调用，从而避免局部的失败和延迟随着一层层的服务依赖扩散、蔓延和逐级放大
  - ✓ 熔断过程：熔断条件监测、切断联系（快速失败或者服务降级）、恢复时机探测、恢复正常

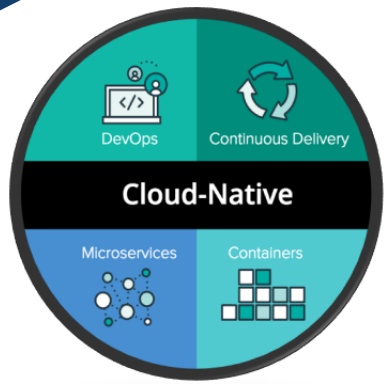
Spring Cloud 提供的Hystrix组件提供了实现超时机制和断路器模式的工具类库，可以用来设置服务调用超时以及实现服务熔断机制

云原生软件技术

- 当前主流的分布式软件系统有很大一部分都是在云上部署和运行的
- “云原生”（Cloud Native）代表顺应云计算特点的软件应用的典型形态
  - ✓ 在云上部署和运行并不代表一个软件系统就完全符合云计算的特点（高度分布式、可伸缩以及资源按需使用等）
  - ✓ 云计算的原住民，即应用的设计、开发、部署和运行方式都充分考虑了云的特点

云原生技术有利于各组织在公有云、私有云和混合云等新型动态环境中，构建和运行可弹性扩展的应用。云原生的代表性技术包括容器、服务网格、微服务、不可变基础设施和声明式API。这些技术能够构建容错性好、易于管理和便于观察的松耦合系统。结合可靠的自动化手段，云原生技术使工程师们能够轻松地对系统作出频繁和可预测的重大变更。 — 云原生计算基金会（CNCF）的定义

云原生软件的特点



据IDC预测，到2022年，90%的应用程序将采用微服务架构；35%的产品应用将是云原生的。

云原生：应用的设计、实现及部署运行方式充分适应云计算平台的特点（如分布式、弹性伸缩）

- ✓ 微服务：物理隔离，轻量级分布式通信，独立部署、独立更新、独立伸缩
- ✓ DevOps：开发运维一体化，持续集成与自动化发布
- ✓ 持续交付：频繁发布、快速交付、快速反馈、快速试错
- ✓ 容器化：服务无差别封装在容器中，运维无需关心服务技术栈

云原生软件的代表性技术-1

- 微服务（Microservice）：应用软件实现为一组独立开发、独立部署、独立伸缩的服务，服务与服务之间实现了松耦合和物理隔离
- 容器（Container）：应用软件服务使用轻量级容器进行打包和部署，使得服务的伸缩更加灵活和快速，而基础设施和资源利用率得以提高
- 不可变基础设施（Immutable Infrastructure）：部署服务的服务器在完成部署之后不再发生变化，直至服务实例被销毁后重新创建（以容器化的方式来对服务及运行环境进行整体构建、打包，同时进行镜像版本

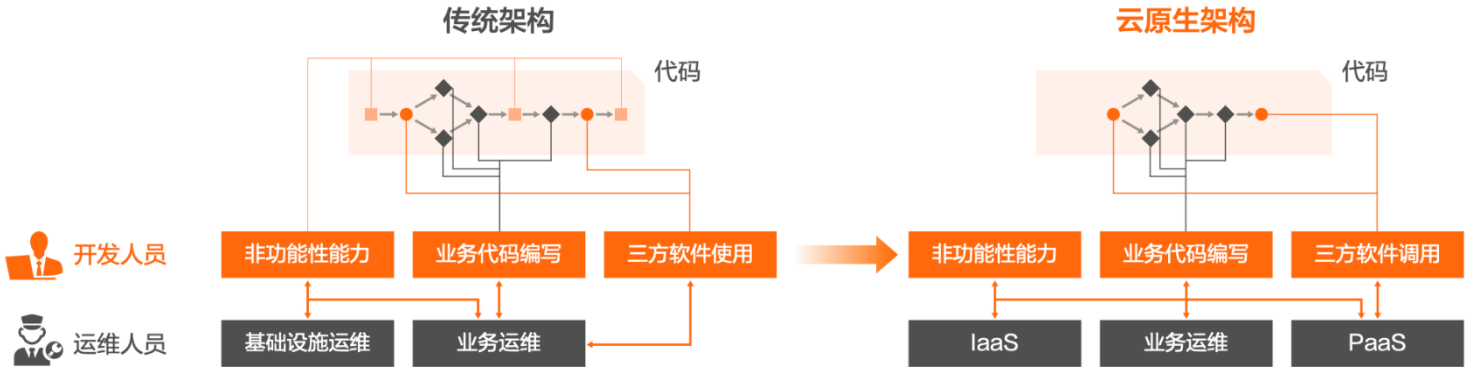
云原生软件的代表性技术-2

- 声明式API（Declarative API）：以声明式的方式来定义运行环境（例如“创建10个X服务的实例”），而不需要通过组合多个命令式的API来定义软件运行所需要的运行环境
- 服务网格（Service Mesh）：应用软件服务之下的一个专门的基础设施层，为每个服务实例提供了轻量级的网络代理，能够在服务开发者无意识的情况下实现服务发现、负载均衡、服务间通信以及相关的可靠性保障（如超时控制、重试熔断等），使得服务开发者可以专注于业务逻辑的实现

云原生软件体系结构

- 出发点：将特定应用业务逻辑实现（如网上购物流程和积分规则等）与通用的技术基础设施（如服务间通信以及服务的部署、伸缩和可用性保障等）相分离
  - ✓ 非功能性质量特性（如性能、可用性、安全性、韧性等）主要由云计算基础设施提供保障
  - ✓ 应用开发者只需要关注于业务视角的体系结构设计（如服务边界划分和服务间交互逻辑设计）

云原生软件的思想中包含一系列软件体系结构设计原则及设计模式，其中体现了模块化、松耦合、关注点分离等经典软件设计思想，同时充分考虑了云计算的特点



阿里云. 云原生架构白皮书

云原生相关的开发过程及管理实践

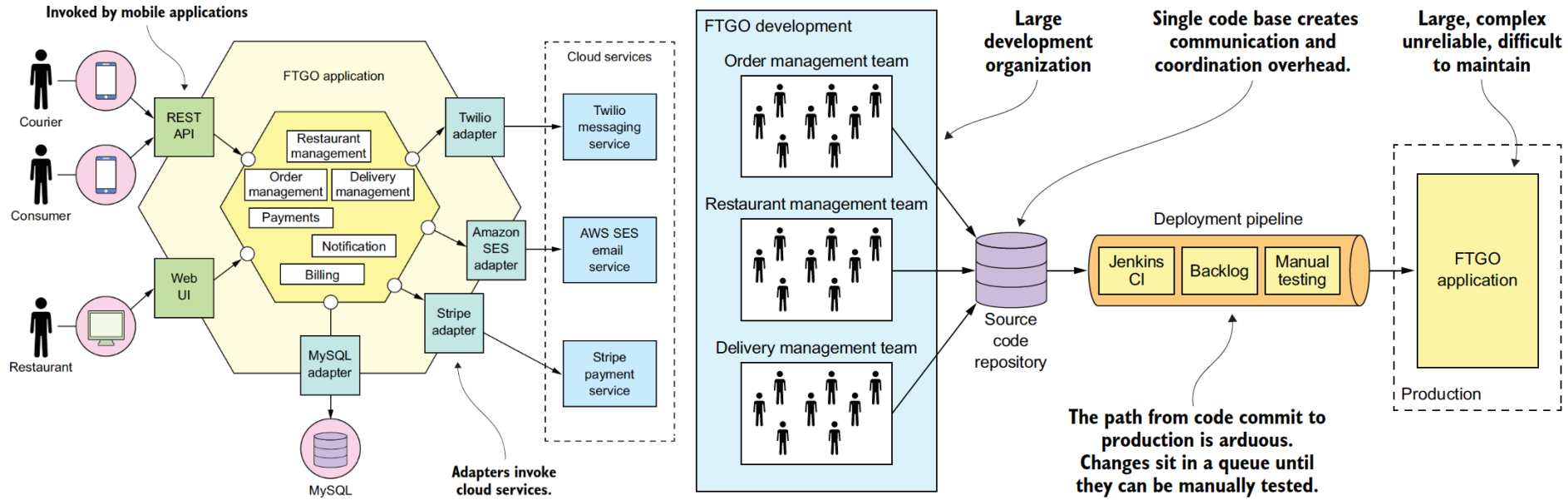
- 开发运维一体化（DevOps）
- 持续交付（Continuous Delivery）
- 康威定律（Conway’s law）
  - ✓ 一个系统的设计结构反映了设计这个系统的企业内部的组织结构和沟通关系
  - ✓ 对于采用微服务架构的云原生软件系统而言，每个服务最好都对应一个相对独立的开发团队，服务与服务之间不仅在软件实现上是松耦合的而且对应的开发职责边界清楚，所需要的开发上的沟通关系与组织结构相一致
  - ✓ 因此，打算采用云原生和微服务架构的企业首先要让开发团队的组织结构与所规划的服务划分结构相适应

单体地狱

单体体系结构（Monolithic Architecture）
又称巨石体系结构

将应用程序构建为单个可执行和可部署组件

代码仓库急剧膨胀，构建和部署变慢，敏捷开发和快速交付很难实现。随着业务的扩张情况还会继续加剧

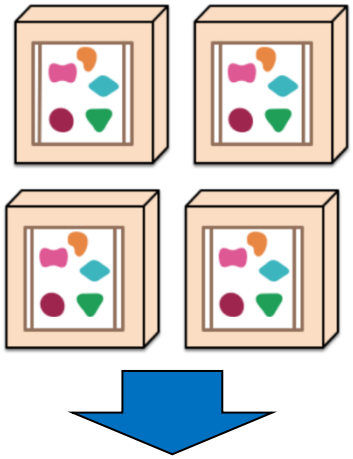


单体体系结构（网络订餐系统）　　单体地狱

Chris Richardson. Microservices Patterns（微服务架构设计模式）. 机械工业出版社（中文版）

从单体到微服务体系结构

单体架构



所有功能在单个进程中实现

以应用为单位进行复制伸缩

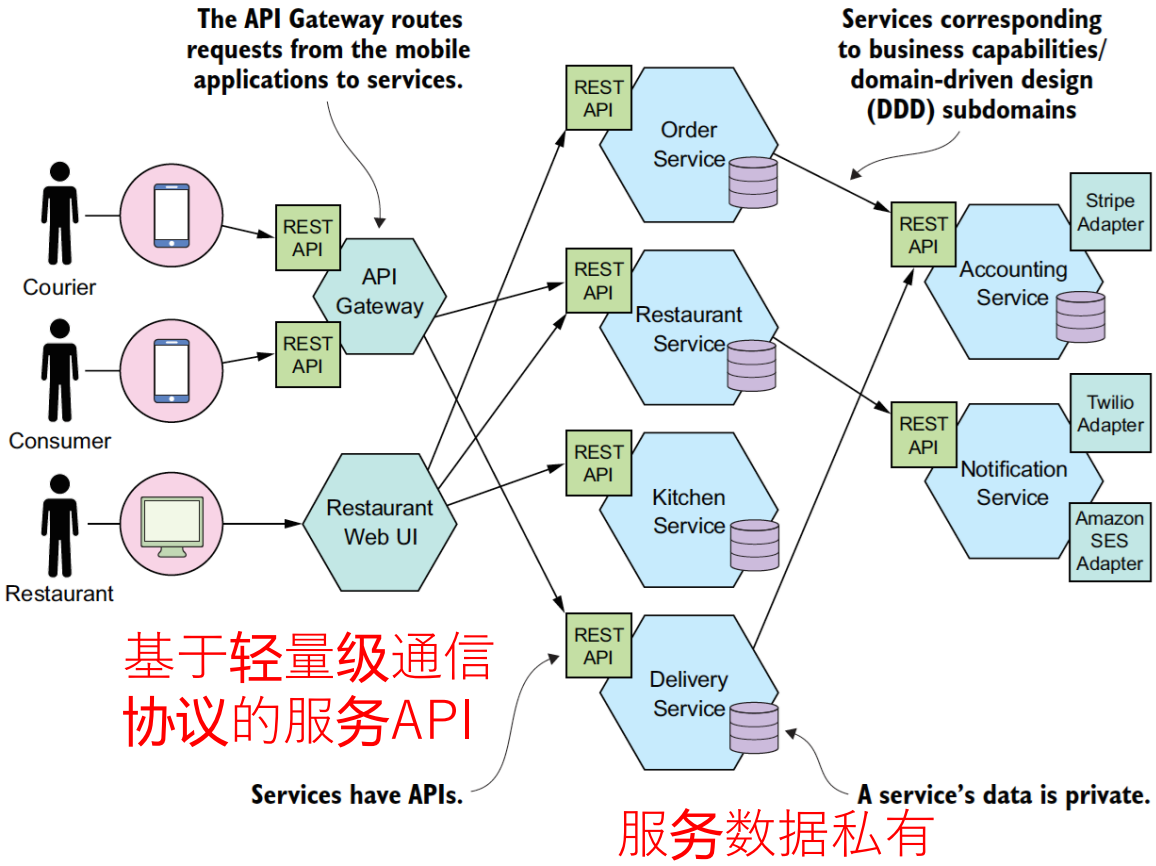
每个功能元素都实现为独立的服务

以服务为单位进行复制伸缩

微服务架构

通过API网关对来自移动应用的服务请求进行路由

围绕业务能力组织服务



基于轻量级通信协议的服务API

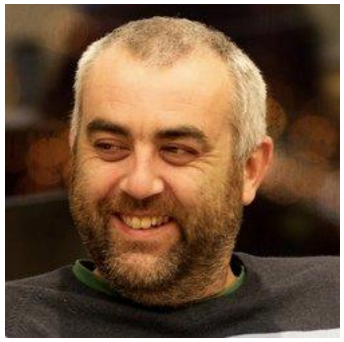
服务数据私有

https://www.martinfowler.com/articles/microservices.html

Chris Richardson. Microservices Patterns（微服务架构设计模式）. 机械工业出版社（中文版）

微服务体系结构的定义

微服务架构风格将单个应用程序实现为一组小的服务，每个服务在自己的进程中运行，并通过轻量级机制进行通信。这些服务是围绕业务功能构建的，可以通过完全自动化的部署机制独立部署。这些服务去集中化管理，可以使用不同的编程语言编写并使用不同的数据存储技术。



Jams Lewis

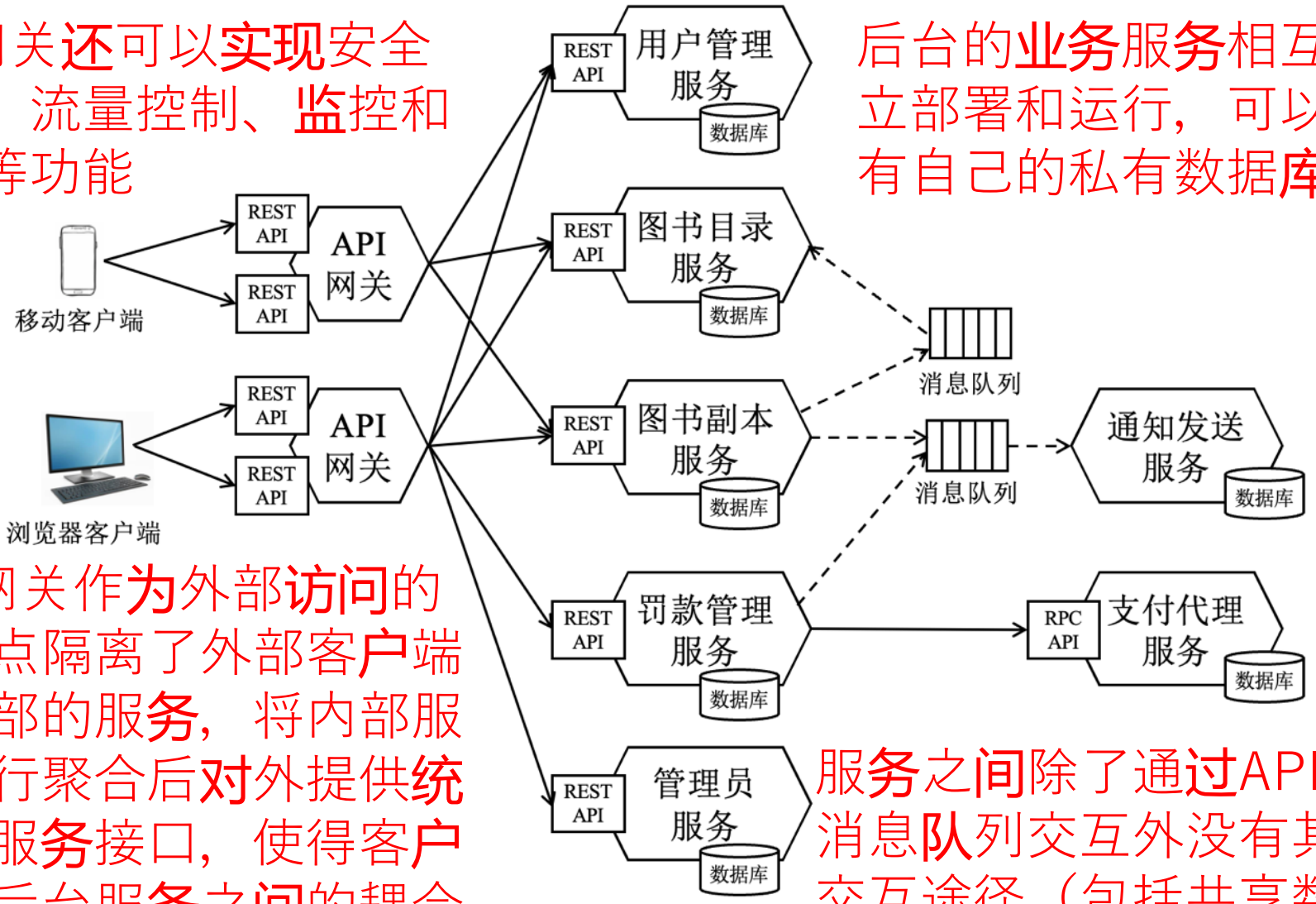


Martin Fowler

图书馆管理子系统的微服务体系结构示例

API网关还可以实现安全防护、流量控制、监控和计费等功能

后台的业务服务相互独立部署和运行，可以拥有自己的私有数据库



API网关作为外部访问的接入点隔离了外部客户端与内部的服务，将内部服务进行聚合后对外提供统一的服务接口，使得客户端与后台服务之间的耦合降低

服务之间除了通过API及消息队列交互外没有其他交互途径（包括共享数据）

微服务开发框架

- 单个服务开发
  - ✓ 总体上与传统Web应用开发类似（甚至可以将单个服务视为一种Web应用）
  - ✓ 服务对外提供的接口（API）以及对其他服务的调用通过REST、RPC等远程调用或基于消息的通信来实现
  - ✓ 服务开发完毕后可以独立打包和部署
- 整体微服务治理结构
  - ✓ 包括一系列微服务体系结构所必需的基础设施，包括服务网关、服务注册发现、负载均衡、容错、配置管理等
  - ✓ 这些服务治理组件自身经常也被实现为服务，并与应用中的业务服务进行交互
- 相关框架同时支持单个服务开发和服务治理
  - ✓ Spring Cloud基于Spring Boot，同时提供了服务治理基础设施
  - ✓ Apache Dubbo提供了面向接口代理的高性能RPC调用
  - ✓ Apache ServiceComb是一个开源的微服务全栈开发框架

本章小结-1

- 软件体系结构是一个软件系统的高层设计结构，决定了系统的高层分解结构以及其他一些最重要的设计决策
  - ✓ 主要挑战在于如何满足非功能性质量需求
  - ✓ 为此需要针对一系列设计问题确定相应的设计决策并决定如何在相互冲突的非功能性质量需求中进行权衡
- 软件体系结构设计一般采用多视图的方式，从设计结构、开发管理、系统集成、安装部署等不同角度进行思考和刻画
- 软件体系结构风格以一种抽象的方式刻画了系统的整体设计样式，为具体的软件体系结构设计提供了参照和指导

本章小结-2

- 与单机部署的软件系统相比，分布式软件系统的体系结构设计具有更大的挑战，需要在考虑分布式通信问题的基础上实现一致性、可用性和容错性等方面的设计目标
- 需要掌握相关设计思想以及进程间通信、负载均衡、分布式存储、可靠性保障等技术
- 云计算技术的发展催生了云原生的概念以及相应的软件技术体系，需要对作为云原生软件主流风格的微服务体系结构以及相应的软件开发框架和容器化部署技术有所了解

现代软件工程基础



End

7. 软件体系结构